Bilkent University Electrical and Electronics Department

EE102-01 Lab 4 Report:

Arithmetic Logic Unit

Muhammed Mesud Bilirgen – 22301965 – 03 March 2025

Purpose

The aim of this lab is to design and implement an Arithmetic Logic Unit (ALU) using VHDL in a modular way. The ALU should perform various arithmetic and logical operations and be deployed on BASYS3 board.

Design Specification

The ALU is designed with the following specifications:

- Inputs: Two 4-bit operands (X, Y) and a 3-bit control signal (OP) to select operations.
- Outputs: 4-bit result (Q) and overflow (O) of the selected operation displayed via FPGA LEDs.
- Operations:
  - Addition (000)
  - Subtraction (001)
  - Decrement (010)
  - Increment (011)
  - One's Complement (100)
  - Two's Complement (101)
  - Arithmetic Shift Left (110)
  - Bitwise XOR (111)

Operations were implemented without using built-in library functions, so basic components were manually designed and reused (Figure 1).

Methodology

The ALU was designed using a modular approach, starting with a half adder as a fundamental building block. Then, a full adder was implemented, followed by a 4-bit adder and an adder-subtractor, each building upon the previous components. For increment, decrement, two’s complement, arithmetic left shift operations, the adder-subtractor component was also utilized. A top-level module was created to integrate all operations to select through a control signal. Test bench were written to simulate the behaviour of the ALU. By doing so, the correctness of operations was ensured. Mapping the inputs to pins and outputs to LEDs of FPGA was required. After adjusting the constraint file accordingly, the ALU was deployed on an FPGA. Finally, it was confirmed that the ALU functioned as expected in hardware and simulation.

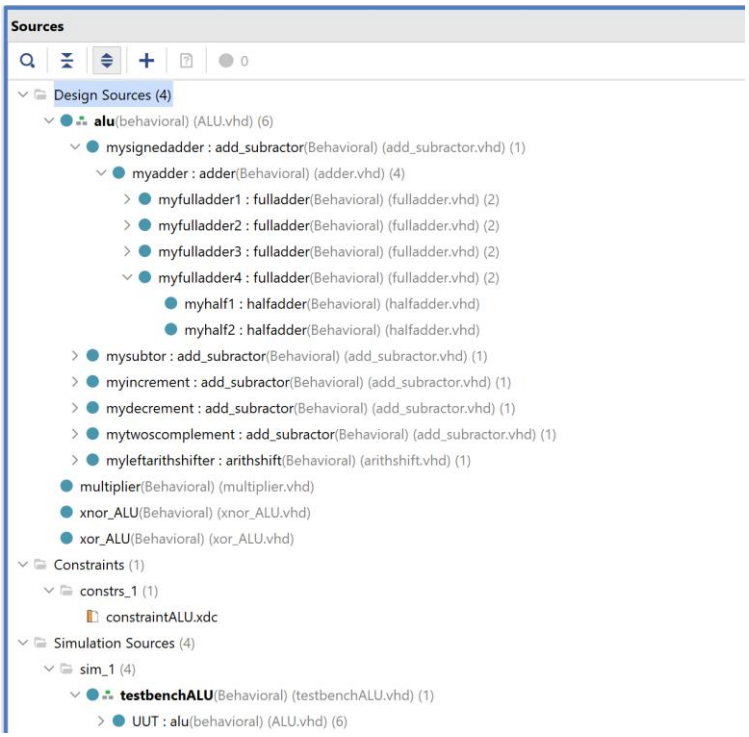

Figure 1: Design sources

Results

The ALU was successfully simulated, and waveforms confirmed correct operation. All cases of inputs for total 11-bits are tested which makes 2048 configuration (Figure 2). An RTL schematic was generated and analysed (Figure 3.1). Moreover, the schematic of add-subtractor component in addition and subtraction operations is as shown (Figure 3.2-3). The ALU was implemented on an FPGA. Some cases for operations are as shown (Figure 4.1-10)

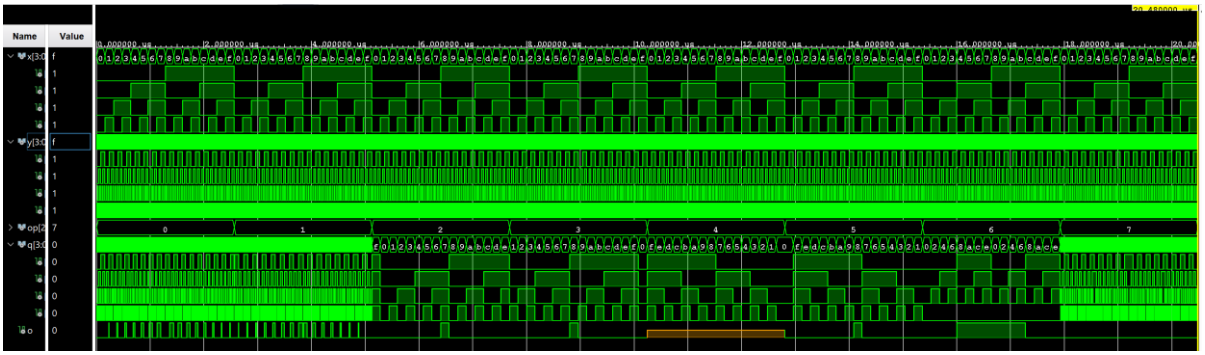

Figure 2: The waveform

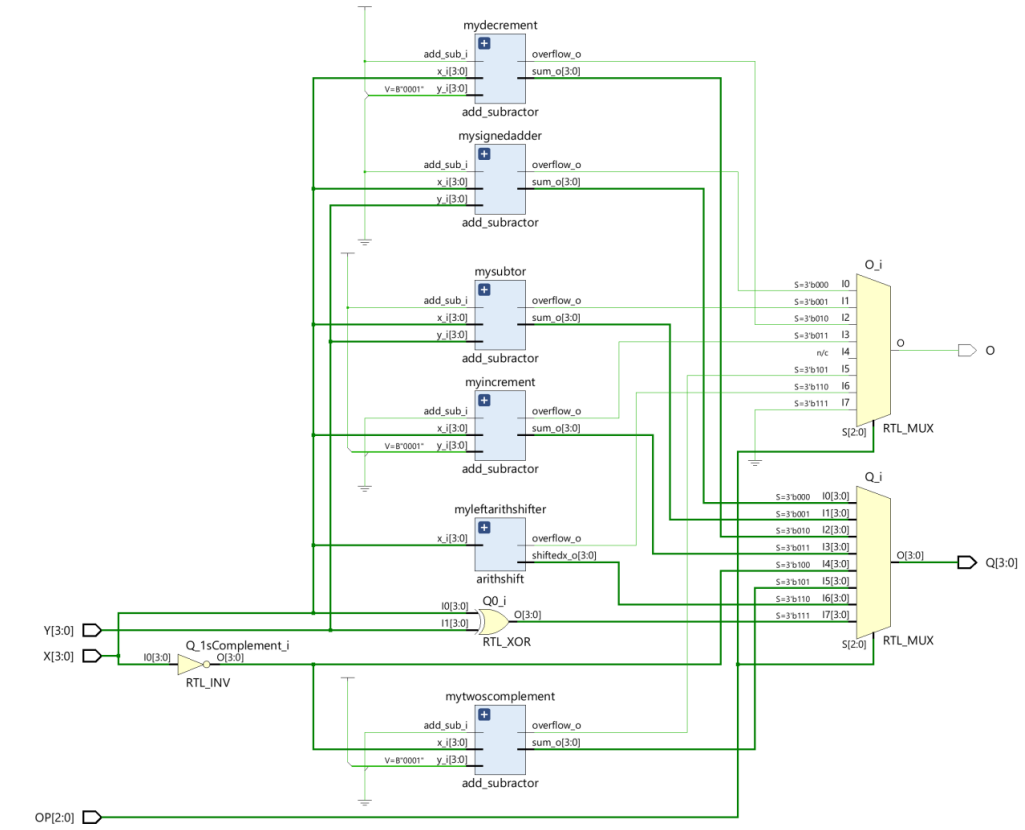

Figure 3.1: The RTL schematic

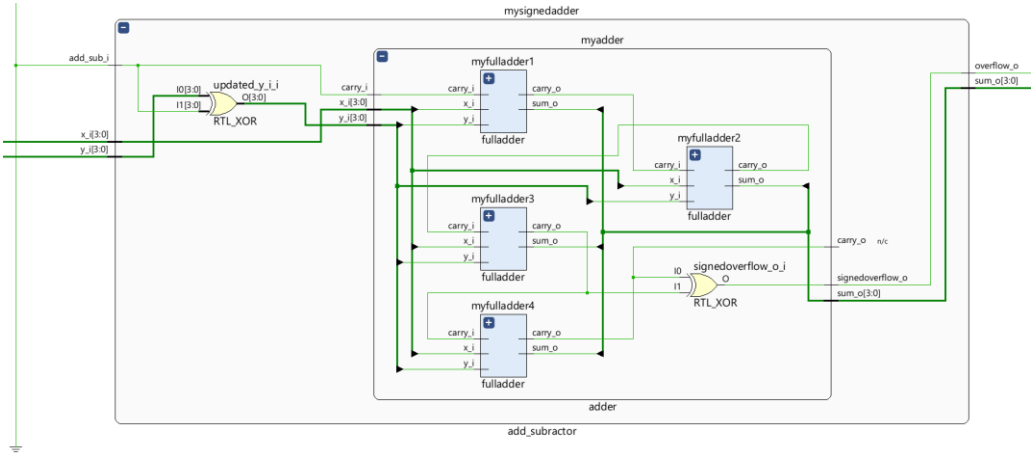

Figure 3.2: The RTL schematic of adder operation using adder-subtractor operation

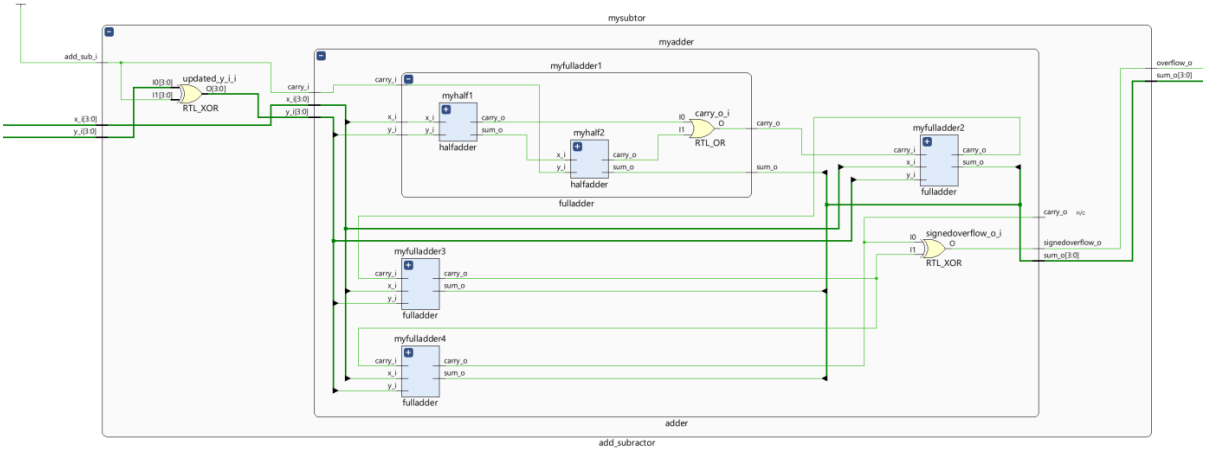

Figure 3.3: The RTL schematic of subtraction operation using adder-subtractor operation

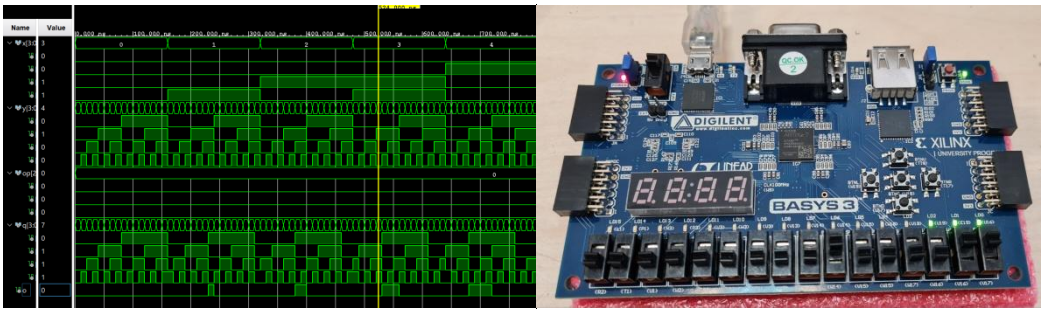

Figure 4.1: Addition operation

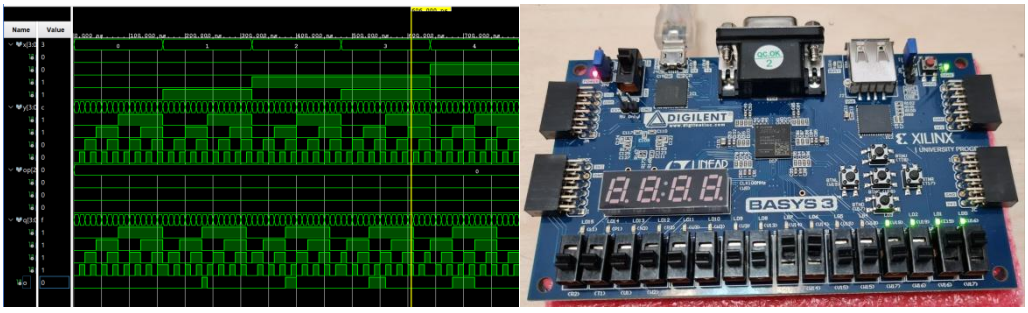

Figure 4.2: Addition operation

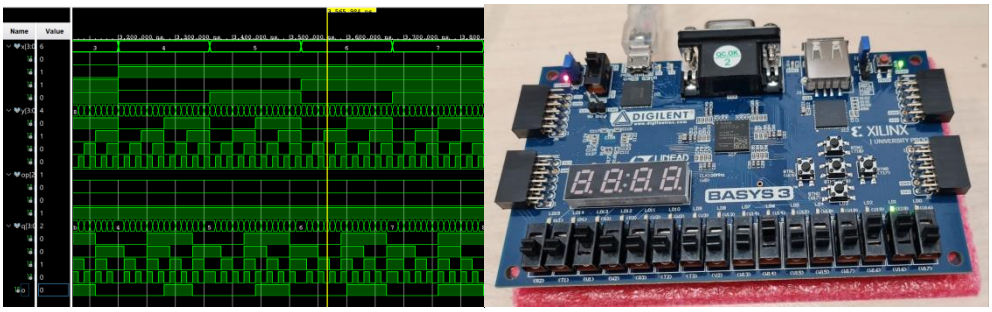

Figure 4.3: Subtraction operation

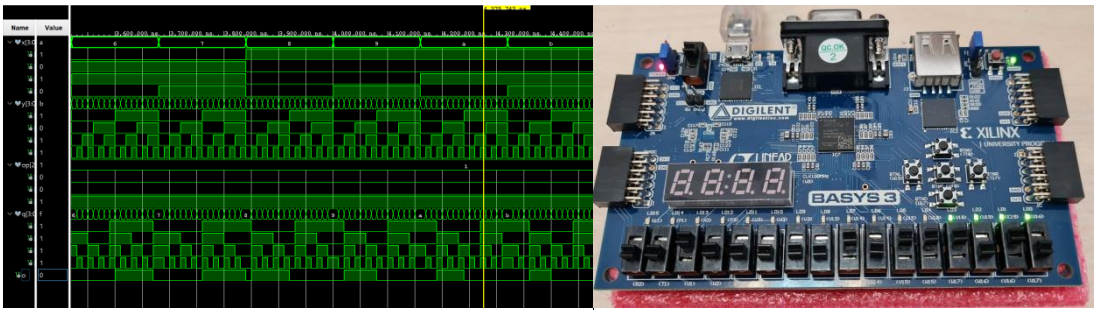

Figure 4.4: Subtraction operation

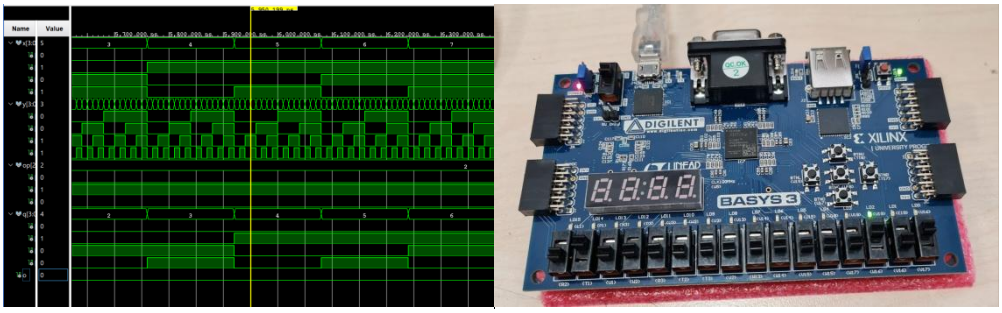

Figure 4.5: Decrement operation

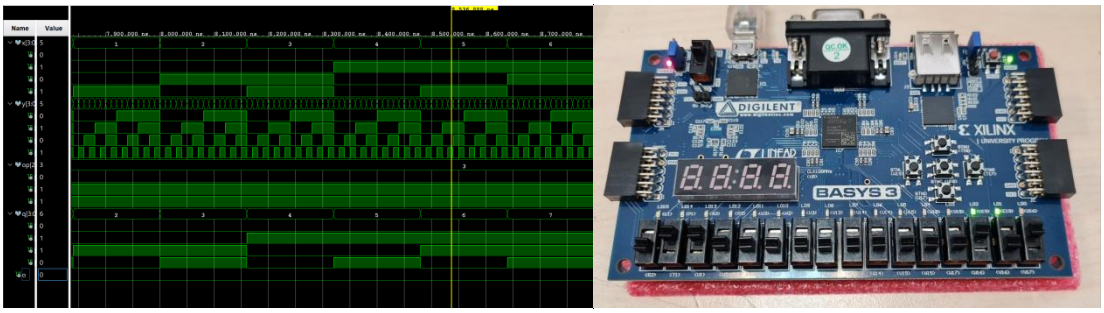

Figure 4.6: Increment operation

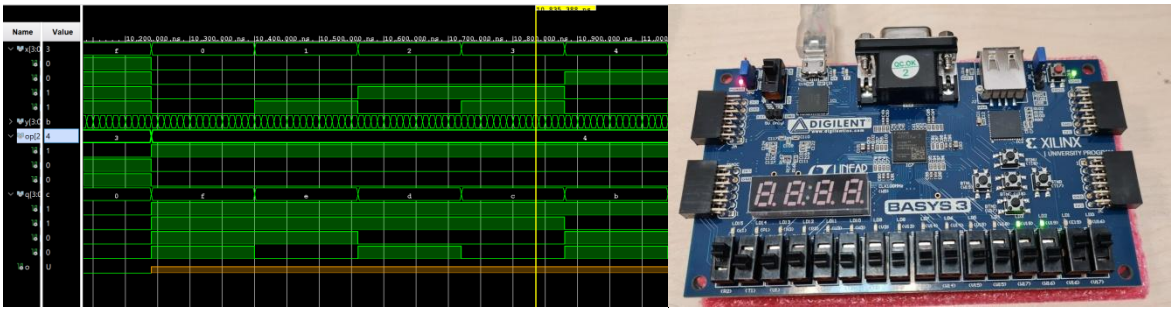

Figure 4.7: Ones complement operation

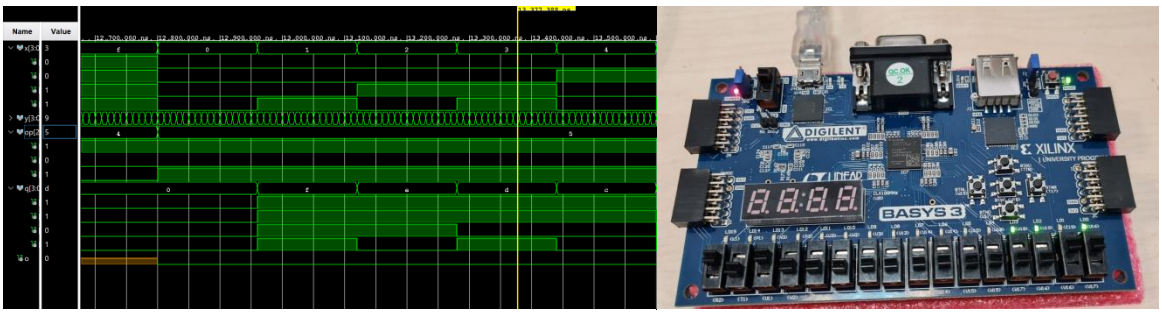

Figure 4.8: Twos complement operation

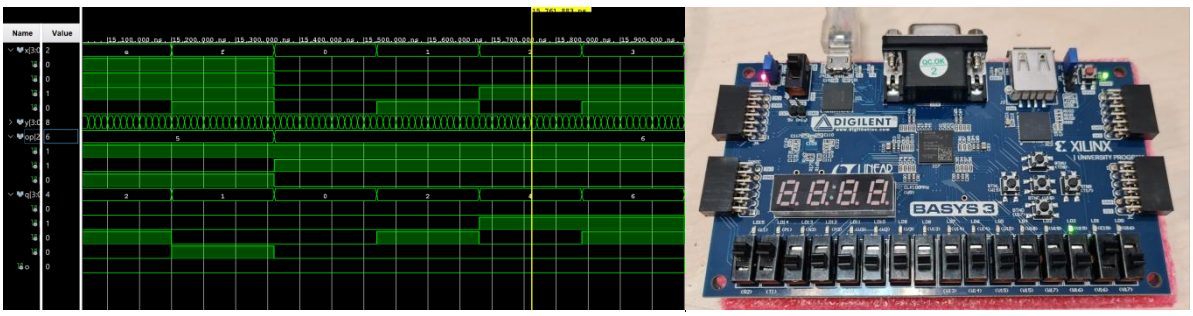

Figure 4.9: Arithmetic left shift operation

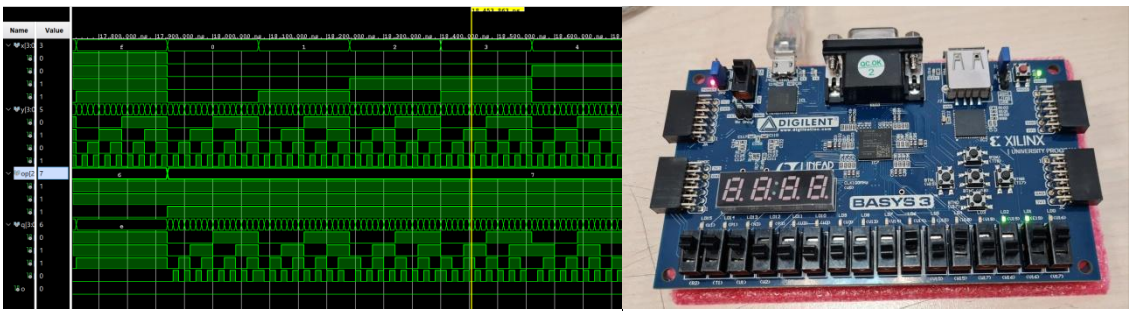

Figure 4.10: Bitwise XOR operation

Conclusion

The experiment was successful to fulfil the purpose. The ALU was designed, simulated, implemented, on BASYS3 board. The design followed a modular approach, constructing the ALU from basic components. The design was verified through simulation and successfully tested on the FPGA. Challenges encountered during debugging VHDL syntax errors and ensuring correct signal assignment.

Appendices

ALU.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity alu is
port (  -- the alu connections to external circuitry:
  X  : in STD_LOGIC_VECTOR(3 downto 0);   -- operand A
  Y  : in STD_LOGIC_VECTOR(3 downto 0);   -- operand B
  OP : in STD_LOGIC_VECTOR(2 downto 0); -- opcode
  Q  : out STD_LOGIC_VECTOR(3 downto 0);  -- operation result
  O  : out STD_LOGIC);
end alu;

architecture behavioral of alu is

signal Q_addition, Q_subraction, Q_increment, Q_decrement, Q_1sComplement, Q_2sComplement,
Q_leftariths : STD_LOGIC_VECTOR (3 downto 0);
signal O_addition, O_subraction, O_increment, O_decrement, O_1sComplement, O_2sComplement,
O_leftariths : STD_LOGIC ;

component add_subractor is
Port ( x_i : in STD_LOGIC_VECTOR (3 downto 0);
       y_i : in STD_LOGIC_VECTOR (3 downto 0);
       overflow_o : out STD_LOGIC;
       sum_o : out STD_LOGIC_VECTOR (3 downto 0);
       add_sub_i : in STD_LOGIC);
end component add_subractor;

component arithshift is
port (x_i : in STD_LOGIC_VECTOR (3 downto 0);
      overflow_o : out STD_LOGIC;
      shiftedx_o : out STD_LOGIC_VECTOR (3 downto 0));
end component arithshift;

begin

mysignedadder : add_subractor
port map(
x_i => X,
y_i => Y,
```

```
overflow_o => O_addition,
sum_o => Q_addition ,
add_sub_i => '0');

mysubtor : add_subractor
port map(
x_i => X,
y_i => Y,
overflow_o => O_subraction,
sum_o => Q_subraction ,
add_sub_i => '1');

myincrement : add_subractor
port map(
x_i => X,
y_i => "0001",
overflow_o => O_increment,
sum_o => Q_increment ,
add_sub_i => '0');

mydecrement : add_subractor
port map(
x_i => X,
y_i => "0001",
overflow_o => O_decrement,
sum_o => Q_decrement ,
add_sub_i => '1');

Q_1sComplement <= not X ;

mytwoscomplement : add_subractor
port map(
x_i => Q_1sComplement,
y_i => "0001",
overflow_o => O_2scomplement,
sum_o => Q_2sComplement,
add_sub_i => '0');

myleftarithshifter : arithshift
port map(
x_i => X,
overflow_o => O_leftariths,
shiftedx_o => Q_leftariths);

process (OP, Q_addition, Q_subraction, Q_decrement, Q_increment,
        Q_1sComplement, Q_2sComplement, Q_leftariths, X, Y,
        O_addition, O_subraction, O_decrement, O_increment,
        O_1sComplement, O_2sComplement, O_leftariths) begin
case OP is  -- decode the opcode and perform the operation:
   when "000" => Q <= Q_addition; O <= O_addition;
   when "001" => Q <= Q_subraction; O <= O_subraction;
   when "010" => Q <= Q_decrement; O <= O_decrement;          -- Decrement X by 1
```

```
   when "011" => Q <= Q_increment;  O <= O_increment;         -- Increment X by 1
   when "100" => Q <= Q_1sComplement; O <= O_1sComplement ;    -- 1's complement
   when "101" => Q <= Q_2sComplement;  O <= O_2sComplement;    -- 2's complement
   when "110" => Q <= Q_leftariths; O <= O_leftariths;        -- Left Arithmetic Shift
   when "111" => Q <= X xor Y;  O <= '0' ;                    -- Bitwise XOR
   when others => Q <= (others => 'X'); O <= '0';             -- Handle undefined cases
end case;

end process;
end behavioral;
```

testbenchALU.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity testbenchALU is
--  Port ( ); --  A testbench does not need ports since it drives and monitors internal signals within the
simulation.
end testbenchALU;

architecture Behavioral of testbenchALU is

   component ALU is
   port(
    X  : in STD_LOGIC_VECTOR(3 downto 0);   -- operand A
    Y  : in STD_LOGIC_VECTOR(3 downto 0);   -- operand B
    OP : in STD_LOGIC_VECTOR(2 downto 0);   -- opcode
    Q  : out STD_LOGIC_VECTOR(3 downto 0);  -- operation result
    O  : out STD_LOGIC);
   end component;

   signal x : std_logic_vector(3 downto 0) := "0000";
   signal y : std_logic_vector(3 downto 0) := "0000";
   signal op : std_logic_vector(2 downto 0) := "000";
   signal q : std_logic_vector(3 downto 0);
   signal o : std_logic;

   begin

   UUT : ALU port map(
   X => x,
   Y => y,
   OP => op,
   Q => q,
   O=> o );

   STIM_PROCESS : process begin
   for k in 0 to 7 loop
   for i in 0 to 15 loop
   for j in 0 to 15 loop
```

```
    OP <= std_logic_vector(to_unsigned(k, 3));
    X <= std_logic_vector(to_unsigned(i, 4));
    Y <= std_logic_vector(to_unsigned(j, 4));

    wait for 10 ns;
   end loop;end loop;end loop;
   assert false report "SIMULATION FINISHED" severity failure;

   end process;
end Behavioral;
```

constraint.vhd

```
# Constraint file sets which input/output in the VHDL code connects to which pin in FPGA.
# Three characters after PACKAGE_PIN gives the FPGA pin which is also written next to all
LEDs/Switches on the board.
# The second line (IOSTANDARD LVCMOS33) sets the voltage standard for the pins which is constant
and always the same for BASYS3.

set_property PACKAGE_PIN V17 [get_ports {X[0]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {X[0]}]
set_property PACKAGE_PIN V16 [get_ports {X[1]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {X[1]}]
set_property PACKAGE_PIN W16 [get_ports {X[2]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {X[2]}]
set_property PACKAGE_PIN W17 [get_ports {X[3]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {X[3]}]

set_property PACKAGE_PIN W15 [get_ports {Y[0]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {Y[0]}]
set_property PACKAGE_PIN V15 [get_ports {Y[1]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {Y[1]}]
set_property PACKAGE_PIN W14 [get_ports {Y[2]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {Y[2]}]
set_property PACKAGE_PIN W13 [get_ports {Y[3]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {Y[3]}]

set_property PACKAGE_PIN U1 [get_ports {OP[0]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {OP[0]}]
set_property PACKAGE_PIN T1 [get_ports {OP[1]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {OP[1]}]
set_property PACKAGE_PIN R2 [get_ports {OP[2]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {OP[2]}]

# LEDs
set_property PACKAGE_PIN U16 [get_ports {Q[0]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {Q[0]}]
set_property PACKAGE_PIN E19 [get_ports {Q[1]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {Q[1]}]
set_property PACKAGE_PIN U19 [get_ports {Q[2]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {Q[2]}]
```

```
set_property PACKAGE_PIN V19 [get_ports {Q[3]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {Q[3]}]

set_property PACKAGE_PIN L1 [get_ports {O}]
        set_property IOSTANDARD LVCMOS33 [get_ports {O}]
```

arithshift.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity arithshift is
port (x_i : in STD_LOGIC_VECTOR (3 downto 0);
    --shiftedamount : in STD_LOGIC_VECTOR (1 downto 0) :="00";
    --arith_logic_i : STD_LOGIC :='0';
    --l_r : in STD_LOGIC :='0';
    overflow_o : out STD_LOGIC;
    shiftedx_o : out STD_LOGIC_VECTOR (3 downto 0));
end arithshift;

architecture Behavioral of arithshift is

component adder is
port (x_i : in STD_LOGIC_VECTOR (3 downto 0);
    y_i : in STD_LOGIC_VECTOR (3 downto 0);
    signedoverflow_o : out STD_LOGIC;
    sum_o : out STD_LOGIC_VECTOR (3 downto 0);
    carry_i : in STD_LOGIC);
end component adder;

begin

myadder : adder
port map(
x_i => x_i ,
y_i => x_i ,
signedoverflow_o => overflow_o ,
sum_o => shiftedx_o ,
carry_i => '0'
);
end Behavioral;
```

add_subractor.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity add_subractor is
Port ( x_i : in STD_LOGIC_VECTOR (3 downto 0);
     y_i : in STD_LOGIC_VECTOR (3 downto 0);
     overflow_o : out STD_LOGIC;
     sum_o : out STD_LOGIC_VECTOR (3 downto 0);
```

```
      add_sub_i : in STD_LOGIC);
end add_subractor;

architecture Behavioral of add_subractor is

component adder is
port (x_i : in STD_LOGIC_VECTOR (3 downto 0);
     y_i : in STD_LOGIC_VECTOR (3 downto 0);
     signedoverflow_o : out STD_LOGIC;
     sum_o : out STD_LOGIC_VECTOR (3 downto 0);
     carry_i : in STD_LOGIC);
end component adder;

signal add_sub_i_vector, updated_y_i : STD_LOGIC_VECTOR (3 downto 0);

begin

myadder : adder
port map(
x_i => x_i ,
y_i => updated_y_i ,
signedoverflow_o => overflow_o ,
sum_o => sum_o ,
carry_i => add_sub_i
);

add_sub_i_vector <= add_sub_i & add_sub_i & add_sub_i & add_sub_i;
updated_y_i <= y_i XOR add_sub_i_vector;

end Behavioral;
```

adder.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity adder is
   Port ( x_i : in STD_LOGIC_VECTOR (3 downto 0);
          y_i : in STD_LOGIC_VECTOR (3 downto 0);
          signedoverflow_o : out STD_LOGIC;
          carry_o : out STD_LOGIC;
          sum_o : out STD_LOGIC_VECTOR (3 downto 0);
          carry_i : in STD_LOGIC);
end adder;

architecture Behavioral of adder is

component fulladder is
Port ( carry_i : in STD_LOGIC;
          x_i : in STD_LOGIC;
          y_i : in STD_LOGIC;
          sum_o : out STD_LOGIC;
```

```
        carry_o : out STD_LOGIC);
end component fulladder;

signal c1,c2,c3,c4          : std_logic;

begin

myfulladder1 : fulladder
port map(
carry_i => carry_i,
x_i => x_i(0),
y_i => y_i(0),
sum_o => sum_o(0),
carry_o => c1);

myfulladder2 : fulladder
port map(
carry_i => c1,
x_i => x_i(1),
y_i => y_i(1),
sum_o => sum_o(1),
carry_o => c2);

myfulladder3 : fulladder
port map(
carry_i => c2,
x_i => x_i(2),
y_i => y_i(2),
sum_o => sum_o(2),
carry_o => c3);

myfulladder4 : fulladder
port map(
carry_i => c3,
x_i => x_i(3),
y_i => y_i(3),
sum_o => sum_o(3),
carry_o => c4);

carry_o <= c4;
signedoverflow_o <= c4 XOR c3;
end Behavioral;
```

fulladder.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity fulladder is
   Port ( carry_i : in STD_LOGIC;
        x_i : in STD_LOGIC;
        y_i : in STD_LOGIC;
```

```
        sum_o : out STD_LOGIC;
        carry_o : out STD_LOGIC);
end fulladder;

architecture Behavioral of fulladder is

component halfadder is
port (
x_i       : in std_logic;
y_i       : in std_logic;
sum_o  : out std_logic;
carry_o : out std_logic);
end component halfadder;

signal sum1,c1,c2          : std_logic;

begin

myhalf1 : halfadder
port map(
x_i => x_i ,
y_i       => y_i ,
sum_o => sum1 ,
carry_o => c1);

myhalf2 : halfadder
port map(
x_i => sum1 ,
y_i       => carry_i ,
sum_o => sum_o ,
carry_o => c2);

carry_o <= c1 OR c2 ;

end Behavioral;
```

halfadder.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity halfadder is
   Port ( x_i : in STD_LOGIC;
        y_i : in STD_LOGIC;
        sum_o : out STD_LOGIC;
        carry_o : out STD_LOGIC);
end halfadder;

architecture Behavioral of halfadder is

begin
```

```
sum_o <= x_i XOR y_i;
carry_o <= x_i AND y_i;

end Behavioral;
```